

گزارش پروژه چهارم بینایی GPS, IMU, EKF

استاد درس: دکتر جوانمردی، دکتر خادمیان

امیررضا کاظم لو

درس ربات‌های متحرک خودگردان

۸ تیر ۱۴۰۵

۱ تبدیل مختصات جغرافیایی به دکارتی (ENU)

برای حل قسمت اول تمرین، ابتدا باید موقعیت‌های GPS را از قالب مختصات جغرافیایی (طول و عرض جغرافیایی) به دستگاه مختصات دکارتی ENU با تقریب یک صفحه محلی تبدیل می‌کردم.

طبق فرض مسئله، زمین را به عنوان یک کره با شعاع ثابت در نظر گرفتیم. شعاع کره زمین در صورت سوال ۶۴۰۰ کیلومتر داده شده است، اما از آنجایی که در مراحل بعدی برای پیاده‌سازی فیلتر کالمن توسعه‌یافته، داده‌های شتاب‌سنج در فایل imu.txt بر حسب واحد m/s^2 هستند، تصمیم گرفتیم از همین ابتدا برای یکپارچگی واحدها، شعاع کره زمین را به متر ($R = 6400000m$) تبدیل کنیم.

با توجه به ساختار فایل gps.txt که شامل زمان، عرض و طول جغرافیایی است، داده‌ها را خوانده و ابتدا تمامی مقادیر طول و عرض جغرافیایی را از درجه به رادیان تبدیل کردم. سپس نقطه اول مجموعه داده را به عنوان مبدا مختصات صفحه محلی یعنی (lat_0, lon_0) حول اولین نقطه در نظر گرفتیم. در نهایت، با استفاده از روابط داده شده، مختصات دکارتی را محاسبه کردم:

$$x_t = R(lon_t - lon_0) \cos(lat_0)$$

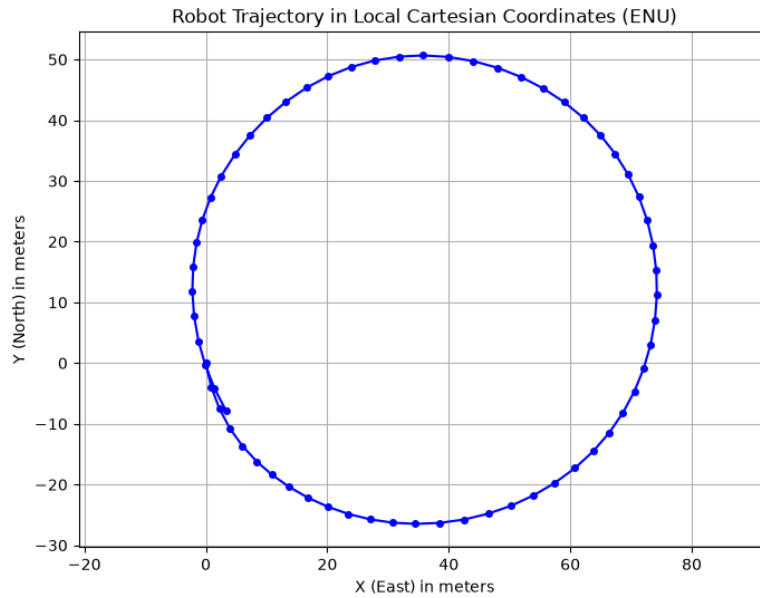
$$y_t = R(lat_t - lat_0)$$

در این روابط، x_t نشان‌دهنده موقعیت در راستای شرق و y_t نشان‌دهنده موقعیت در راستای شمال در صفحه تقریب زده شده است. برای پیاده‌سازی این بخش از کتابخانه pandas برای بارگذاری داده‌ها و از numpy برای محاسبات سریع و برداری روی کل داده‌ها استفاده کردم.

جهت صحت‌سنجی درستی محاسبات و کدهای نوشته شده، مسیر حرکت ربات را روی صفحه دو بعدی رسم کردم. همان‌طور که در شکل ۱ مشخص است، مسیر تبدیل‌شده به درستی یک حرکت دایره‌ای را نشان می‌دهد که از نقطه مبدا یعنی مختصات $(0, 0)$ آغاز شده است.

۲ مدل تغییر حالت و مشاهدات

برای استخراج مدل سینماتیک ربات، باید معادلات حرکت را بر اساس متغیرهای حالت، داده‌های سنسور IMU و مشاهدات GPS فرمول‌بندی می‌کردم. طبق صحبت‌های دکتر خادمیان به این مدل کردن می‌گفت loosely coupled



شکل ۱: مسیر حرکت ربات در دستگاه مختصات محلی دکارتی (ENU)

یعنی ما دیتای خام از سنسور ها دریافت نمی کنیم و به فیلتر مون بدیم بلکه مختصات دقیق را در سنسور ها (این که کجا هستیم) می گیریم.
طبق راهنمایی اول تمرین، بردار حالت سیستم را ۵ بعدی در نظر گرفتیم. این بردار شامل موقعیت، سرعت خطی در دستگاه جهانی و زاویه سمت (Heading) ربات است:

$$X_t = \begin{bmatrix} x_t \\ y_t \\ \dot{x}_t \\ \dot{y}_t \\ \psi_t \end{bmatrix}$$

ورودی های سیستم نیز از فایل IMU استخراج می شوند که شامل شتاب های خطی در راستای محورهای بدنه $(\ddot{x}_B, \ddot{y}_B)$ و سرعت زاویه ای حول محور قائم بدنه $(\dot{\psi})$ هستند:

$$U_t = \begin{bmatrix} \ddot{x}_B \\ \ddot{y}_B \\ \dot{\psi} \end{bmatrix}$$

۱.۲ مدل System Equation

نکته مهم در تشکیل مدل تغییر حالت به فرم $X_t = g(X_{t-1}, U_t, \Delta t) + \epsilon$ این است که شتاب سنج، مقادیر را در دستگاه مختصات بدنه (Body Frame) اندازه گیری می کند، در حالی که متغیرهای موقعیت و سرعت در بردار حالت در دستگاه مختصات جهانی (Global Frame) قرار دارند. بنابراین، ابتدا شتاب های بدنه را با استفاده از زاویه ψ_{t-1} به

شتاب‌های جهانی ($\ddot{x}_G, \ddot{y}_G$) تصویر کردم:

$$\ddot{x}_G = \ddot{x}_B \cos(\psi_{t-1}) - \ddot{y}_B \sin(\psi_{t-1})$$

$$\ddot{y}_G = \ddot{x}_B \sin(\psi_{t-1}) + \ddot{y}_B \cos(\psi_{t-1})$$

سپس با استفاده از معادلات پایه سینماتیک، مدل نهایی تغییر حالت را به صورت زیر به دست آوردم:

$$X_t = \begin{bmatrix} x_{t-1} + \dot{x}_{t-1}\Delta t + \frac{1}{2}(\ddot{x}_B \cos \psi_{t-1} - \ddot{y}_B \sin \psi_{t-1})\Delta t^2 \\ y_{t-1} + \dot{y}_{t-1}\Delta t + \frac{1}{2}(\ddot{x}_B \sin \psi_{t-1} + \ddot{y}_B \cos \psi_{t-1})\Delta t^2 \\ \dot{x}_{t-1} + (\ddot{x}_B \cos \psi_{t-1} - \ddot{y}_B \sin \psi_{t-1})\Delta t \\ \dot{y}_{t-1} + (\ddot{x}_B \sin \psi_{t-1} + \ddot{y}_B \cos \psi_{t-1})\Delta t \\ \psi_{t-1} + \dot{\psi}\Delta t \end{bmatrix} + \epsilon$$

۲.۲ مدل Observation Model

بر اساس راهنمایی دوم تمرین، موقعیت تبدیل‌شده GPS (مؤلفه‌های x و y) را مستقیماً به عنوان مشاهده در نظر گرفتیم. از آنجا که مشاهدات صرفاً استخراج مستقیم دو متغیر اول از بردار حالت هستند، مدل کاملاً خطی خواهد بود. بردار مشاهدات ۲ بعدی به فرم زیر است:

$$Y_t = \begin{bmatrix} x_{gps} \\ y_{gps} \end{bmatrix}$$

مدل مشاهدات را به صورت $Y_t = HX_t + \delta$ تعریف کردم که در آن H ماتریس مشاهدات با ابعاد ۲ در ۵ است. این ماتریس فقط x_t و y_t را استخراج می‌کند:

$$Y_t = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_t \\ y_t \\ \dot{x}_t \\ \dot{y}_t \\ \psi_t \end{bmatrix} + \delta$$

البته بد نیست اینجا به یک نکته هم اشاره کنم؛ همان‌طور که سر کلاس دکتر خادهمیان مطرح کردند، ایده جالب‌تری هم برای مدل مشاهدات وجود داشت. می‌توانستم شتاب و سرعت زاویه‌ای رو با استفاده از تفاضل‌گیری مقادیر حالت در دو یا سه گام زمانی قبل (Time Steps) تخمین بزنم و آن‌ها رو به عنوان متغیرهای تاخیردار توی مشاهدات قرار بدم. این کار قاعدتاً به فیلتر کمک می‌کرد تا تاریخچه حرکت ربات رو بهتر درک کنه و کمی از اثر نویزهای لحظه‌ای کم بشه.

اما خب، از آنجایی که توی راهنمایی دوم صورت سوال پیشنهاد شده بود که فقط موقعیت GPS رو مستقیم به عنوان مشاهده در نظر بگیرم تا مدل خطی بمونه، من هم تصمیم گرفتم مسیر منطبق بر خواسته سوال رو انتخاب کنم.

۳ ماتریس ژاکوبین مدل تغییر حالت

برای قدم بعدی در فیلتر کالمن توسعه‌یافته (EKF)، باید مدل غیرخطی تغییر حالت رو حول نقطه فعلی (یعنی تخمین قبلی سیستم) خطی‌سازی می‌کردم. این کار با محاسبه ماتریس ژاکوبین نسبت به بردار حالت X_{t-1} انجام می‌شود.

محاسبه این ماتریس برای آپدیت کردن ماتریس کواریانس خطا در مرحله Prediction مهم است، چون میزان عدم قطعیت مدل ما رو تو هر گام زمانی مشخص می‌کنه.

با توجه به تابع $g(X_{t-1}, U_t, \Delta t)$ که در بخش قبل به دست آوردیم، ماتریس ژاکوبین 5×5 در 5 به صورت زیر تعریف می‌شود:

$$F_t = \frac{\partial g}{\partial X_{t-1}} = \begin{bmatrix} \frac{\partial x_t}{\partial x_{t-1}} & \frac{\partial x_t}{\partial y_{t-1}} & \frac{\partial x_t}{\partial \dot{x}_{t-1}} & \frac{\partial x_t}{\partial \dot{y}_{t-1}} & \frac{\partial x_t}{\partial \psi_{t-1}} \\ \frac{\partial y_t}{\partial x_{t-1}} & \frac{\partial y_t}{\partial y_{t-1}} & \frac{\partial y_t}{\partial \dot{x}_{t-1}} & \frac{\partial y_t}{\partial \dot{y}_{t-1}} & \frac{\partial y_t}{\partial \psi_{t-1}} \\ \frac{\partial \dot{x}_t}{\partial x_{t-1}} & \frac{\partial \dot{x}_t}{\partial y_{t-1}} & \frac{\partial \dot{x}_t}{\partial \dot{x}_{t-1}} & \frac{\partial \dot{x}_t}{\partial \dot{y}_{t-1}} & \frac{\partial \dot{x}_t}{\partial \psi_{t-1}} \\ \frac{\partial \dot{y}_t}{\partial x_{t-1}} & \frac{\partial \dot{y}_t}{\partial y_{t-1}} & \frac{\partial \dot{y}_t}{\partial \dot{x}_{t-1}} & \frac{\partial \dot{y}_t}{\partial \dot{y}_{t-1}} & \frac{\partial \dot{y}_t}{\partial \psi_{t-1}} \\ \frac{\partial \psi_t}{\partial x_{t-1}} & \frac{\partial \psi_t}{\partial y_{t-1}} & \frac{\partial \psi_t}{\partial \dot{x}_{t-1}} & \frac{\partial \psi_t}{\partial \dot{y}_{t-1}} & \frac{\partial \psi_t}{\partial \psi_{t-1}} \end{bmatrix}$$

با مشتق‌گیری جزئی از تک‌تک معادلات تغییر حالت نسبت به متغیرهای $[x, y, \dot{x}, \dot{y}, \psi]$ ، ماتریس نهایی رو به این شکل محاسبه کردم:

$$F_t = \begin{bmatrix} 1 & 0 & \Delta t & 0 & \frac{1}{2}(-\ddot{x}_B \sin \psi_{t-1} - \ddot{y}_B \cos \psi_{t-1})\Delta t^2 \\ 0 & 1 & 0 & \Delta t & \frac{1}{2}(\ddot{x}_B \cos \psi_{t-1} - \ddot{y}_B \sin \psi_{t-1})\Delta t^2 \\ 0 & 0 & 1 & 0 & (-\ddot{x}_B \sin \psi_{t-1} - \ddot{y}_B \cos \psi_{t-1})\Delta t \\ 0 & 0 & 0 & 1 & (\ddot{x}_B \cos \psi_{t-1} - \ddot{y}_B \sin \psi_{t-1})\Delta t \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

همان‌طور که در صورت تمرین هم به درستی پیشنهاد شده بود، برای پیاده‌سازی راحت‌تر و اصولی‌تر این بخش توی کدم، یک تابع پایتون مجزا تعریف کردم. کار این تابع اینه که هر بار مقادیر نقطه مشتق‌گیری (یعنی تخمین فعلی حالت و شتاب‌های خوانده‌شده از IMU) رو به همراه اختلاف زمان Δt بگیره و این ماتریس 5×5 در 5 رو خروجی بده. اینطوری هر بار که فیلتر کالمن در مرحله Prediction فراخوانی میشه، ماتریس ژاکوبین به‌صورت کاملاً دینامیک و بر اساس جدیدترین وضعیت ربات محاسبه و جایگذاری میشه. این ماتریس را در تابع `calculate_jacobian_F` پیاده سازی کردم. ورودی‌های تابع مقادیر زیر هستند:

• `X_prev (numpy.ndarray)`: بردار حالت قبلی $[x, y, v_x, v_y, \psi]$

• `U_t (numpy.ndarray)`: بردار ورودی فعلی $[a_{x_B}, a_{y_B}, \omega]$

• `dt (float)`: گام زمانی (Δt)

۴ پیاده‌سازی مرحله Update

همان‌طور که در صورت تمرین اشاره شده بود، از آنجا که موقعیت گرفته‌شده از GPS را مستقیماً به عنوان مشاهده در نظر گرفتیم، مدل مشاهدات من کاملاً خطی است. این خطی بودن کار را بسیار ساده می‌کند، زیرا ماتریس مشاهدات (H) یک ماتریس ثابت است و در مرحله به‌روزرسانی نیازی به خطی‌سازی مجدد و محاسبه ژاکوبین برای بخش مشاهدات ندارم. و معادلات آن خطی کالمن استاندارد می‌شود.

ابتدا Innovation را که تفاضل بین مقدار اندازه‌گیری‌شده توسط سنسور و مقدار پیش‌بینی‌شده توسط مدل است، محاسبه کردم:

$$y_t = Z_t - HX_{pred}$$

سپس ماتریس کواریانس باقیمانده (S_t) را به دست آوردم. در فرمول زیر Q نویز مشاهده است:

$$S_t = HP_{pred}H^T + Q_{obs}$$

در قدم بعدی، Kalman Gain را محاسبه کردم تا مشخص شود چقدر باید به مشاهدات جدید در مقابل پیش‌بینی مدل اعتماد کنم:

$$K_t = P_{pred}H^T S_t^{-1}$$

در نهایت، با استفاده از بهره‌ی کالمن، بردار حالت و ماتریس کواریانسِ belief سیستم را به‌روزرسانی کردم:

$$X_{upd} = X_{pred} + K_t y_t$$

$$P_{upd} = (I - K_t H) P_{pred}$$

در کد این بخش‌ها در تابع `update(X_pred, P_pred, Z_meas, H, Q_obs)` پیاده‌سازی شده است که

$$P_{upd}, X_{upd}$$

را در خروجی دارد.